LEARNING NOTES / QKSITFALL

Quantum EV Charging

From booking rules to QAOA

A software engineer's guide to the actual Python implementation

This document explains the complete project README, including the mathematical steps hidden inside the implementation. It begins with ordinary booking data, builds the optimization objective, translates it into quantum operations, and explains what the measurements mean.

No previous quantum mechanics is assumed. Symbols are introduced before use; longer derivations, a glossary, and exercises with answers appear in the appendices.

Execution model

Qiskit QAOA executed on a local Aer simulator. Classical Python prepares the problem and adjusts circuit angles. No IBM API key is used by the current command. **Code snapshot**

1 November 2026. Qiskit 2.6.1; Qiskit Aer 0.18.2; SciPy 1.18.1. The numerical examples follow `main.py` and `src/quantum_scheduler.py`.

Contents

1 How to use this guide **1**

2 The charging problem before quantum computing **1**

2.1 The input scenario . . . 1

2.2 Energy, power, and arrival . . . 2

2.3 Individually feasible bookings . . . 2

3 Checkpoint 1: turn possible bookings into bits **3**

4 Checkpoint 2: build the QUBO objective **3**

4.1 Two reusable scheduling rules . . . 3

4.2 The actual sample equation . . . 4

4.3 Choosing the penalty without solving the schedule . . . 4

4.4 How the builder stores the equation . . . 4

5 Quantum vocabulary, introduced from zero **5**

5.1 A bit, a qubit, a gate, and a measurement . . . 5

5.2 Superposition and phase . . . 5

5.3 Many qubits and the meaning of a bit string . . . 5

5.4 Operators, eigenvalues, and two different meanings of H . . . 6

6 Checkpoint 3a: translate binary costs into an Ising operator **6**

6.1 Pauli Z and the identity operator . . . 6

6.2 One cost term, step by step . . . 6

6.3 Translate every linear and pair term . . . 7

6.4 The sample's actual operator . . . 7

6.5 Offset and normalization . . . 7

7 Checkpoint 3b: build the actual QAOA gates **8**

7.1 Initialize with Hadamard gates . . . 8

7.2 Cost operations change phases . . . 8

7.3 Why RZ and RZZ have these angles . . . 8

7.4 Mixing turns phase differences into probability changes . . . 9

7.5 A one-qubit demonstration before five-qubit scheduling . . . 9

7.6 Layers, gate order, and the constant offset . . . 9

8 Checkpoint 4: train the circuit angles **10**

8.1 What the optimizer minimizes . . . 10

8.2 What COBYLA changes . . . 10
8.3 A concrete training trace . . . 11

9 Checkpoint 5: measure, decode, and check 11
9.1 Sampling the trained circuit . . . 11
9.2 Qiskit bit order . . . 11
9.3 Validity and sampling metrics . . . 12

10 Checkpoint 6: choose a sampled schedule and report it 12

11 Experiments: what to improve and how to measure it 13

12 Boundaries, hardware access, and the browser explainer 14
12.1 What the current physical model assumes . . . 14
12.2 Simulation versus a physical quantum processor . . . 14
12.3 The interactive parking illustration . . . 14
12.4 Code-to-concept map . . . 15

A Notation and glossary 16

B Mathematical background and gate calculations 18
B.1 Complex numbers and magnitudes . . . 18
B.2 Relative phase can be revealed by a later gate . . . 18
B.3 Tensor products and Qiskit's basis order . . . 18
B.4 Matrix forms of the rotations . . . 19
B.5 The one-qubit phase/mixer example, fully worked . . . 19

C Full numerical audit of the sample objective 20
C.1 Expanded QUBO coefficients . . . 20
C.2 Derive one Z coefficient and one pair coefficient . . . 20
C.3 Verify the selected schedule three ways . . . 20
C.4 General coefficient generation and a matrix convention trap . . . 20

D Exercises and common points of confusion 21
D.1 Interpretations to keep separate . . . 21

E Exercise answers 22

F Commands, defaults, and reproducibility 23

References 24

1 How to use this guide

Read Sections 2–4 first if the scheduling model or penalty expressions are unclear. Read Sections 5–7 slowly for the transition from bits to gates. Then run the program while reading Sections 8–11.

The implemented workflow is

$$\text{vehicle, station, and travel data} \longrightarrow \text{possible bookings} \longrightarrow \text{binary objective}$$
$$\longrightarrow \text{quantum cost operator} \longrightarrow \text{QAOA circuit} \longrightarrow \text{measured booking bits.}$$

The quantum circuit participates in searching for a schedule. It does not calculate road distances, parse vehicle data, or invent the scheduling rules.

Run these commands from the repository root:

```
.venv/bin/python main.py
.venv/bin/python main.py --layers 1 --show-circuit
.venv/bin/python -m unittest discover -s tests -v
```

The main entry point prints six quantum checkpoints. The document follows those checkpoints, with additional foundations where they are needed.

> **Key idea.** There are three different objects to keep separate: a real booking, the binary variable that selects it, and the qubit used to represent that variable.

2 The charging problem before quantum computing

2.1 The input scenario

We coordinate two cars and two stations. Each station has one charging port with power 11 kW. Each reservation occupies an entire one-hour slot: 08:00–09:00 or 09:00–10:00.

Vehicle property	Car A	Car B
Battery capacity	50 kWh	50 kWh
Initial charge	30%	12%
Target charge	50%	30%
Consumption	0.2 kWh/km	0.2 kWh/km
Available to depart	07:45	07:50
Charging deadline	10:00	09:00

Journey	A–North	A–South	B–North	B–South
Distance (km)	2	5	1	20
Duration (min)	10	15	5	30

Electricity at North costs Rs 2/kWh in the early slot and Rs 1/kWh in the late slot. South costs Rs 3/kWh and Rs 2/kWh respectively. “Rs” denotes rupees; we keep monetary amounts separate from dimensionless optimization score points.

2.2 Energy, power, and arrival

Let B_v be vehicle v's battery capacity in kWh; s_v and t_v its initial and target state-of-charge *fractions*; d_{vs} its distance to station s; and e_v its consumption in kWh/km. The Python inputs store percentages, so 30% is entered as `30` and divided by 100 in the calculation.

$$E_v^{\text{initial}} = B_v s_v, \tag{2.1}$$
$$E_{vs}^{\text{drive}} = d_{vs} e_v, \tag{2.2}$$
$$E_{vs}^{\text{arrival}} = E_v^{\text{initial}} - E_{vs}^{\text{drive}}, \tag{2.3}$$
$$D_{vs} = B_v t_v - E_{vs}^{\text{arrival}}. \tag{2.4}$$

Here D_{vs} is the charging energy required *at that station*. A farther station can require more charging, because the drive reduces the arrival energy.

For A driving to North:

$$\begin{aligned} E_A^{\text{initial}} &= 50(0.30) = 15.0\,\text{kWh}, \\ E_{A,N}^{\text{drive}} &= 2(0.2) = 0.4\,\text{kWh}, \\ E_{A,N}^{\text{arrival}} &= 15.0 - 0.4 = 14.6\,\text{kWh}, \\ D_{A,N} &= 50(0.50) - 14.6 = 10.4\,\text{kWh}. \end{aligned}$$

If station power is W_s kW, ideal charging duration is

$$\tau_{vs} = 60\frac{D_{vs}}{W_s} \quad \text{minutes}. \tag{2.5}$$

Thus A needs $60(10.4/11) \approx 56.73$ minutes at North. Energy in kWh is an amount; power in kW is an amount delivered per hour.

2.3 Individually feasible bookings

The function `generate_options()` examines every vehicle–station–slot combination. A booking survives if all four checks pass:

1. Arrival battery is at least the reserve, $0.05B_v$ by default.
2. Arrival time is no later than the selected slot's start.
3. Starting then, actual charging completes no later than the vehicle deadline.
4. The required energy fits within the slot's charging capacity.

For B at South, arrival energy would be $6 - 20(0.2) = 2\,\text{kWh}$, below the 2.5 kWh reserve. B also cannot take North's late slot because charging would finish after its 09:00 deadline. Only North's early slot survives for B.

For booking i, define electricity bill b_i, waiting time w_i in minutes, and booking score c_i:

$$b_i = D_{vs}\, r_{s,k}, \qquad w_i = T_k^{\text{start}} - T_{vs}^{\text{arrival}}, \qquad c_i = b_i + \lambda w_i, \tag{2.6}$$

where $r_{s,k}$ is the electricity tariff and $\lambda = 0.25$ is the waiting weight. Numerically, each rupee contributes one score point and each waiting minute contributes 0.25 points. The score is a preference, not a physical law.

3 Checkpoint 1: turn possible bookings into bits

The five surviving options are assigned consecutive indices by `build_qubo()`. Each $x_i \in \{0, 1\}$ selects one booking. Its corresponding qubit is q_i.

Bit	Booking	Bill (Rs)	Wait (min)	c_i
x_0	A, North, 08:00	20.80	5	22.05
x_1	A, North, 09:00	10.40	65	26.65
x_2	A, South, 08:00	33.00	0	33.00
x_3	A, South, 09:00	22.00	60	37.00
x_4	B, North, 08:00	18.40	5	19.65

All five variables appear in the objective, but a valid schedule selects only one booking for each car. The vector

$$\boldsymbol{x} = (0, 1, 0, 0, 1) \tag{3.1}$$

selects A at North late and B at North early. Their reservations occur at different times. We plan both cars jointly because their options share resources.

The groups are built from data:

```
groups = {"A": [0, 1, 2, 3], "B": [4]}
```

This displayed dictionary is the sample result, not a hardcoded assumption in the builder. Adding cars or options changes the groups and the number of qubits. B's forced booking could be fixed classically, but the current example retains its qubit to make the rule application uniform.

4 Checkpoint 2: build the QUBO objective

QUBO stands for *quadratic unconstrained binary optimization*. "Binary" means bits, "quadratic" means terms involving at most pairs of bits, and "unconstrained" means the scheduling rules have been incorporated as penalties inside the objective. It does not mean the scheduling problem has no rules.

4.1 Two reusable scheduling rules

Let $\mathcal{V}$ be the vehicle set, G_v the indices of bookings belonging to vehicle v, and $\mathcal{C}$ the set of unordered pairs of bookings for different vehicles that occupy the same station and slot. The general objective is

$$f(\boldsymbol{x}) = \underbrace{\sum_i c_i x_i}_{\text{booking scores}} + \underbrace{P \sum_{v \in \mathcal{V}} \left(\sum_{i \in G_v} x_i - 1 \right)^2}_{\text{one booking per car}} + \underbrace{P \sum_{(i,j) \in \mathcal{C}} x_i x_j}_{\text{no shared charger conflicts}} . \tag{4.1}$$

The sum over a car counts its selected bookings. Its penalty is zero exactly when that count equals one. A conflict product is one only if both conflicting bookings are selected. These templates apply to any number of cars and options under the current single-port, single-slot-session model.

Selected bookings for one car	0	1	2	3
Exactly-one penalty	P	0	P	$4P$

4.2 The actual sample equation

Here $G_A = \{0, 1, 2, 3\}$, $G_B = \{4\}$, and $\mathcal{C} = \{(0, 4)\}$. Therefore

$$\begin{aligned} f(\boldsymbol{x}) = {} & 22.05x_0 + 26.65x_1 + 33x_2 + 37x_3 + 19.65x_4 \\ & + P(x_0 + x_1 + x_2 + x_3 - 1)^2 \\ & + P(x_4 - 1)^2 + Px_0x_4. \end{aligned} \tag{4.2}$$

The first penalty belongs to A; the second belongs to B; the last belongs to their shared North/early resource. Giving B more options changes its group sum. Another conflict creates another pair term automatically.

For the valid vector $(0, 1, 0, 0, 1)$ every penalty is zero:

$$f(0, 1, 0, 0, 1) = 26.65 + 19.65 = 46.30. \tag{4.3}$$

For $(1, 0, 0, 0, 1)$ both cars receive a booking, but both use North early:

$$f(1, 0, 0, 0, 1) = 22.05 + 19.65 + P = 41.70 + P. \tag{4.4}$$

The per-car penalties alone would not detect this charger conflict.

4.3 Choosing the penalty without solving the schedule

All booking scores are nonnegative. Define the upper bound

$$U = \sum_{v \in \mathcal{V}} \max_{i \in G_v} c_i, \qquad P = U + 1. \tag{4.5}$$

Any valid schedule selects one option per car and therefore scores at most U. Any invalid bit string incurs at least P in penalties plus nonnegative booking cost. If a valid schedule exists, no invalid string can be the global minimizer. This statement is about the objective, not a promise that QAOA finds its minimum.

For the sample, $U = 37 + 19.65 = 56.65$ and $P = 57.65$. A custom smaller penalty may make invalid strings attractive. Increasing P far beyond necessity can make the differences between valid operating costs harder to resolve during training.

4.4 How the builder stores the equation

Using $x_i^2 = x_i$ for bits, the builder expands every group penalty:

$$P\left(\sum_{i \in G_v} x_i - 1\right)^2 = P - P\sum_{i \in G_v} x_i + 2P \sum_{\substack{i<j \\ i,j \in G_v}} x_i x_j. \tag{4.6}$$

The stored representation is

$$f(\boldsymbol{x}) = C_0 + \sum_i L_i x_i + \sum_{i<j} Q_{ij} x_i x_j. \tag{4.7}$$

The code names are `constant`, `linear`, and `quadratic`. Here $C_0 = P|\mathcal{V}|$, each $L_i = c_i - P$, each same-car pair gets $2P$, and each different-car resource conflict gets P. Negative L_i values are a consequence of expanding penalties; they are not negative electricity bills.

> **Check your understanding.** If a third car has bookings x_5 and x_6, write its exactly-one penalty. If x_1 and x_5 share a charger and slot, write their conflict penalty. Answers appear in Appendix E.

5 Quantum vocabulary, introduced from zero

5.1 A bit, a qubit, a gate, and a measurement

A classical bit has value 0 or 1. A qubit has two distinguished states, written $|0\rangle$ and $|1\rangle$. The vertical bar and angle bracket form a *ket*: notation for a state vector. They do not mean absolute value here. In column-vector notation,

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \tag{5.1}$$

A general pure one-qubit state is

$$|\psi\rangle = a|0\rangle + b|1\rangle, \qquad |a|^2 + |b|^2 = 1. \tag{5.2}$$

The Greek letter ψ ("psi") names the state. The complex numbers a, b are *amplitudes*. Measurement in this basis produces ordinary bit 0 with probability $|a|^2$, or bit 1 with probability $|b|^2$. The squared magnitude is why an amplitude is not itself a probability.

A *gate* is an allowed transformation of the state, represented by a unitary matrix. A *circuit* is a sequence of gates. Measurement converts the final state into a classical outcome. A *shot* is one preparation, execution, and measurement of the circuit.

5.2 Superposition and phase

The state

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \tag{5.3}$$

has equal measurement probabilities. Its two amplitudes have magnitude $1/\sqrt{2}$, whose square is $1/2$. The square root and normalization matter.

An amplitude can be written $a = re^{\mathrm{i}\phi}$, where $r \geq 0$ is its magnitude, ϕ is its phase angle in radians, and $\mathrm{i}^2 = -1$. Then $|a|^2 = r^2$. An arrow in a plane is a useful picture: length represents magnitude and direction represents phase. Changing only direction does not change length. This is a mathematical picture, not a physical arrow in the chip.

Relative phases matter when later operations combine amplitude contributions. Contributions may reinforce or cancel; this is *interference*. For example, $(|0\rangle + |1\rangle)/\sqrt{2}$ and $(|0\rangle - |1\rangle)/\sqrt{2}$ give the same probabilities if measured immediately, but a subsequent Hadamard gate distinguishes them. Appendix B works this out explicitly.

5.3 Many qubits and the meaning of a bit string

With n qubits, a state is

$$|\psi\rangle = \sum_{\boldsymbol{x}\in\{0,1\}^n} a_{\boldsymbol{x}}|\boldsymbol{x}\rangle, \qquad \sum_{\boldsymbol{x}} |a_{\boldsymbol{x}}|^2 = 1. \tag{5.4}$$

Here $|\boldsymbol{x}\rangle$ denotes the basis state in which qubit q_i stores bit x_i. The logical vector is ordered $(x_0, \ldots, x_{n-1})$; Qiskit's *displayed strings* reverse that order, as explained later. There are 2^n possible outcomes: $2^5 = 32$ in our direct five-qubit encoding. One shot returns one outcome, not a readable list of all 32 amplitudes.

An *entangled* state cannot be written as a product of independent single-qubit states. The RZZ interactions in our cost circuit can generate entanglement. Entanglement by itself does not certify an optimization benefit.

5.4 Operators, eigenvalues, and two different meanings of H

An *operator* is a linear transformation of state vectors, often written as a matrix. If an operator $\widehat{A}$ acts on a state $|u\rangle$ as

$$\widehat{A}|u\rangle = \lambda|u\rangle, \tag{5.5}$$

then $|u\rangle$ is an *eigenvector* and λ its *eigenvalue*. The output points along the same vector, multiplied by a number. This gives a precise meaning to saying that a cost operator "associates a score" with a definite booking string.

We distinguish the Hadamard gate H_{Had} from the cost Hamiltonian $\widehat{H}_C$. They share a conventional letter but are different objects. A Hamiltonian is a Hermitian operator; a gate is unitary. We use the Hamiltonian to construct gates rather than applying its generally nonunitary matrix as a gate.

6 Checkpoint 3a: translate binary costs into an Ising operator

6.1 Pauli Z and the identity operator

The identity operator $\mathbb{I}$ leaves every state unchanged. The Pauli Z operator is

$$\widehat{Z} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad \widehat{Z}|0\rangle = +|0\rangle, \qquad \widehat{Z}|1\rangle = -|1\rangle. \tag{6.1}$$

Its eigenvalues on the two bit states are $+1$ and -1. Applying Z to a superposition changes the sign of the $|1\rangle$ amplitude; it does not flip a bit. The operator $\widehat{Z}_i$ acts on qubit i and as the identity on other qubits.

The classical sign z_i associated with a definite bit is

$$z_i = (-1)^{x_i}, \qquad x_i = \frac{1 - z_i}{2}. \tag{6.2}$$

Booking bit x_i	Sign z_i	$(1 - z_i)/2$
0	$+1$	0
1	-1	1

The quantum counterpart of the binary variable is the projector

$$\widehat{x}_i = \frac{\mathbb{I} - \widehat{Z}_i}{2}. \tag{6.3}$$

It has eigenvalue zero on bit 0 and one on bit 1. Keeping x_i, z_i, and $\widehat{Z}_i$ distinct prevents a common confusion: a bit value is not a matrix.

6.2 One cost term, step by step

Consider only $f(x) = 20x$. It assigns scores zero and twenty. Substituting gives

$$20x = 10 - 10z, \qquad \widehat{H}_f = 10\mathbb{I} - 10\widehat{Z}. \tag{6.4}$$

Thus $\widehat{H}_f|0\rangle = 0|0\rangle$ and $\widehat{H}_f|1\rangle = 20|1\rangle$. The scoring function is unchanged; the same scores are now eigenvalues of an operator. In this context, "energy" means optimization score, not the EV's battery energy in kWh.

6.3 Translate every linear and pair term

For the expanded QUBO in Equation (4.7),

$$L_i x_i \longmapsto \frac{L_i}{2}\mathbb{I} - \frac{L_i}{2}\widehat{Z}_i, \tag{6.5}$$

$$Q_{ij} x_i x_j \longmapsto \frac{Q_{ij}}{4}\left(\mathbb{I} - \widehat{Z}_i - \widehat{Z}_j + \widehat{Z}_i\widehat{Z}_j\right). \tag{6.6}$$

All terms can be collected as

$$\widehat{H}_f = C_I\mathbb{I} + \widehat{H}_C, \qquad \widehat{H}_C = \sum_i h_i\widehat{Z}_i + \sum_{i<j} J_{ij}\widehat{Z}_i\widehat{Z}_j, \tag{6.7}$$

with coefficients

$$C_I = C_0 + \frac{1}{2}\sum_i L_i + \frac{1}{4}\sum_{i<j} Q_{ij}, \tag{6.8}$$

$$h_i = -\frac{L_i}{2} - \frac{1}{4}\sum_{j\neq i} Q_{\{i,j\}}, \tag{6.9}$$

$$J_{ij} = \frac{Q_{ij}}{4}. \tag{6.10}$$

In Equation (6.9), $Q_{\{i,j\}}$ means the stored coefficient for the unordered pair, using its smaller index first. An absent pair contributes zero. This is precisely the bookkeeping in `qubo_to_ising()`.

The *Ising form* uses signs ± 1 and their pair products. The name comes from a model of interacting spins in physics. We use its mathematical structure; no knowledge of physical magnets is required to formulate our application.

6.4 The sample's actual operator

The calculated single-qubit coefficients are

$$(h_0, h_1, h_2, h_3, h_4) = (-83.0875, -70.975, -74.15, -76.15, 4.5875). \tag{6.11}$$

Every pair within $\{0,1,2,3\}$ has $J_{ij} = 28.825$, and $J_{04} = 14.4125$. All other pair coefficients vanish. The offset is $C_I = 227.7125$.

$$\begin{aligned}\widehat{H}_C = {} & -83.0875\widehat{Z}_0 - 70.975\widehat{Z}_1 - 74.15\widehat{Z}_2 \\ & - 76.15\widehat{Z}_3 + 4.5875\widehat{Z}_4 \\ & + 28.825\sum_{0\le i<j\le 3}\widehat{Z}_i\widehat{Z}_j + 14.4125\widehat{Z}_0\widehat{Z}_4.\end{aligned} \tag{6.12}$$

Appendix C verifies these numbers against the original QUBO. The Qiskit class `SparsePauliOp` stores this weighted sum without constructing a full dense 32×32 matrix. "Sparse" means representing the listed terms; it does not mean that the simulator's full statevector stops growing with 2^n.

6.5 Offset and normalization

The circuit uses the normalized nonconstant cost

$$s = \max(\{|h_i|\}_i \cup \{|J_{ij}|\}_{i<j}), \qquad \widetilde{H}_C = \frac{\widehat{H}_C}{s}. \tag{6.13}$$

For this sample, $s = 83.0875$. The implementation uses $s = 1$ if all coefficients vanish. Positive scaling and adding constants preserve the ordering of scores. The reported expectation is restored by

$$\mathbb{E}[f] = s\,\langle \widetilde{H}_C \rangle + C_I. \tag{6.14}$$

The optimizer receives $\langle \widetilde{H}_C \rangle$; the printed history uses Equation (6.14). These are different numerical scales for the same optimization landscape.

7 Checkpoint 3b: build the actual QAOA gates

7.1 Initialize with Hadamard gates

The single-qubit Hadamard gate is

$$H_{\text{Had}} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad H_{\text{Had}}|0\rangle = |+\rangle. \tag{7.1}$$

Starting with all qubits in zero, apply one H gate per qubit:

$$|\psi_0\rangle = |+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{\boldsymbol{x}} |\boldsymbol{x}\rangle. \tag{7.2}$$

The tensor product $\otimes$ combines systems. Initially each five-bit outcome has probability 1/32. Initialization does not know which schedule is good.

```
circuit.h(range(circuit.num_qubits))
```

7.2 Cost operations change phases

For one layer the cost operation is

$$U_C(\gamma) = \exp\left(-\mathrm{i}\gamma \widetilde{H}_C\right). \tag{7.3}$$

The real number γ ("gamma") is adjustable. If $\widehat{H}_C|\boldsymbol{x}\rangle = E_C(\boldsymbol{x})|\boldsymbol{x}\rangle$, then

$$U_C(\gamma)|\boldsymbol{x}\rangle = e^{-\mathrm{i}\gamma E_C(\boldsymbol{x})/s}|\boldsymbol{x}\rangle. \tag{7.4}$$

The phase factor has magnitude one. It changes the amplitude's direction, not its magnitude, so the cost layer alone leaves computational-basis probabilities unchanged. A higher score does not automatically become a lower probability.

All our Z and ZZ terms commute, meaning their operator products can be reordered without changing the result. Consequently Equation (7.3) factors exactly into one gate per term; no approximation from splitting these commuting terms is required.

7.3 Why RZ and RZZ have these angles

Qiskit's rotation conventions are[2, 3]

$$R_Z(\theta) = e^{-\mathrm{i}\theta \widehat{Z}/2}, \tag{7.5}$$

$$R_{ZZ}(\theta) = e^{-\mathrm{i}\theta \widehat{Z}_i \widehat{Z}_j/2}. \tag{7.6}$$

The symbol θ ("theta") is an angle in radians. One full turn is 2π radians; $\pi/2$ radians is 90°. Matching the exponents gives

$$e^{-\mathrm{i}\gamma h_i \widehat{Z}_i/s} = R_{Z,i}\left(\frac{2\gamma h_i}{s}\right), \tag{7.7}$$

$$e^{-\mathrm{i}\gamma J_{ij} \widehat{Z}_i \widehat{Z}_j/s} = R_{ZZ,ij}\left(\frac{2\gamma J_{ij}}{s}\right). \tag{7.8}$$

This is the source of the factor of two in the code. There is no new vehicle rule hidden in the gate angle; it is a convention of the gate definition.

```
circuit.rz(2 * gamma[layer] * coefficient / scale, qubit)
circuit.rzz(2 * gamma[layer] * coefficient / scale, i, j)
```

For example, $h_0 = -s$, so the first qubit's RZ angle is simply -2γ.

7.4 Mixing turns phase differences into probability changes

Pauli X flips the two basis states:

$$\widehat{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \widehat{X}|0\rangle = |1\rangle, \quad \widehat{X}|1\rangle = |0\rangle. \tag{7.9}$$

The mixer Hamiltonian and its operation are

$$\widehat{H}_M = \sum_i \widehat{X}_i, \qquad U_M(\beta) = e^{-\mathrm{i}\beta \widehat{H}_M} = \prod_i R_{X,i}(2\beta). \tag{7.10}$$

The real number β ("beta") controls mixing. RX combines the amplitudes for basis states differing in one bit. Depending on their phases, contributions reinforce or cancel. Useful angles must be learned; arbitrary mixing can make the distribution worse.

```
for qubit in range(circuit.num_qubits):
    circuit.rx(2 * beta[layer], qubit)
```

7.5 A one-qubit demonstration before five-qubit scheduling

Consider again $f(x) = 20x$. Ignore normalization for this small derivation, so the nonconstant cost is $-10\widehat{Z}$. Choose $\gamma = \pi/40$ and $\beta = -\pi/4$. The corresponding rotations are $R_Z(-\pi/2)$ and $R_X(-\pi/2)$.

Circuit so far	Pr(0)	Pr(1)
H_{Had}	1/2	1/2
$R_Z(-\pi/2)H_{\mathrm{Had}}$	1/2	1/2
$R_X(-\pi/2)R_Z(-\pi/2)H_{\mathrm{Had}}$	1	0

These angles are selected for the demonstration. They are not a general solution for the EV instance. Appendix B supplies the amplitude calculation.

7.6 Layers, gate order, and the constant offset

For p layers, begin with $|\psi_0\rangle$ and repeat

$$|\psi_\ell\rangle = U_M(\beta_\ell)U_C(\gamma_\ell)|\psi_{\ell-1}\rangle, \qquad \ell = 1, \ldots, p. \tag{7.11}$$

Operators on the right act first: cost, then mixing. With $p = 2$ there are four adjustable angles. Python indexes layers from zero, so mathematical γ_1 is `gamma[0]`. The Hadamard initialization appears only once. An *ansatz* is this chosen parameterized circuit structure.

Including the offset $C_I\mathbb{I}$ would multiply the whole state by $e^{-\mathrm{i}\gamma C_I/s}$, a common or *global* phase. It cancels from all measurement probabilities. Omitting it saves work without losing the cost information needed to rank outcomes. We retain C_I to restore reported scores.

> **Check your understanding.** After H and the cost layer, why can all five-bit probabilities still be 1/32? Which operation allows phase differences to influence those probabilities?

8 Checkpoint 4: train the circuit angles

8.1 What the optimizer minimizes

For angles $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_p)$ and $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_p)$, let

$$p_{\beta,\gamma}(\boldsymbol{x}) = |\langle \boldsymbol{x} \mid \psi_p(\boldsymbol{\beta}, \boldsymbol{\gamma})\rangle|^2 . \tag{8.1}$$

The bra $\langle \boldsymbol{x}|$ is the conjugate transpose of the corresponding ket. The inner product extracts that outcome's amplitude. Squaring its magnitude gives the probability. Appendix B explains the notation in detail.

The expected QUBO score is the probability-weighted average

$$F(\boldsymbol{\beta}, \boldsymbol{\gamma}) = \sum_{\boldsymbol{x}} p_{\beta,\gamma}(\boldsymbol{x}) f(\boldsymbol{x}) = s\langle \psi_p | \widetilde{H}_C | \psi_p \rangle + C_I. \tag{8.2}$$

The sum includes invalid booking strings and their penalties. A circuit can occasionally produce an excellent valid schedule while still having a poor average because many other measurements would be invalid.

The implementation passes the normalized expectation to COBYLA:

```
pub = (circuit, normalized_cost, angles)
energy = float(
    estimator.run([pub], precision=0).result()[0].data.evs
)
expected_score = energy * scale + offset
```

`precision=0` asks the local Aer estimator for the ideal expectation, computed to numerical precision without finite-shot sampling noise[4]. The quantum state is still simulated on a classical computer. Final measurements are sampled separately; `-shots` does not change the training precision.

8.2 What COBYLA changes

COBYLA is a derivative-free classical numerical optimizer[5]. It proposes circuit angles, receives an expectation, and proposes new angles. It does not choose booking bits directly and does not call an exhaustive scheduling solver in the quantum path.

$$\text{angles} \longrightarrow \text{Aer circuit evaluation} \longrightarrow \text{expectation} \longrightarrow \text{new angles.}$$

The implementation uses `maxiter` as an objective-evaluation budget per restart, initial step size `rhobeg=0.5`, and trust-region tolerance `tol=1e-4`. The CLI requires at least $2p + 2$ evaluations. Convergence may occur before the budget is exhausted; no global optimum is certified.

Different restarts use different initial angles. The seed initializes a local random generator, and the best parameter vector actually evaluated across all restarts is retained. With $p = 2$, Qiskit's parameter order for this circuit is

$$(\beta_1, \beta_2, \gamma_1, \gamma_2). \tag{8.3}$$

In Python these are `beta[0]`, `beta[1]`, `gamma[0]`, and `gamma[1]`. This is the binding order, not the execution order of the gates. Use the returned `parameter_order` instead of guessing if the circuit changes.

8.3 A concrete training trace

For the current code and default seed, the three restarts used 92, 146, and 78 evaluations respectively, for 316 total. The best expected score observed was approximately

$$F_{\text{best}} \approx 73.88034. \tag{8.4}$$

This is not the final selected schedule's score of 46.30. Equation (8.2) explains the difference: the expectation averages the whole distribution. The pure uniform starting state would have mean score $C_I = 227.7125$, because all nonidentity Z and ZZ expectations vanish. A restart's first evaluated angles are random, nonzero angles; its initial expectation need not equal that uniform-state mean.

9 Checkpoint 5: measure, decode, and check

9.1 Sampling the trained circuit

After binding the best angles, append measurements and execute the circuit:

```
measured = circuit.assign_parameters(best_angles)
measured.measure_all()
counts = simulator.run(
    measured, shots=4096, seed_simulator=7
).result().get_counts()
```

Each dictionary entry is a displayed bit string and the number of shots that produced it. The entries count circuit outcomes, not optimizer evaluations.

9.2 Qiskit bit order

Logical booking vectors in this guide use $(x_0, x_1, x_2, x_3, x_4)$. Qiskit displays the classical bits in reverse index order:

$$\text{displayed string} = x_4 x_3 x_2 x_1 x_0. \tag{9.1}$$

Thus the displayed string `10010` corresponds to

$$(x_0, x_1, x_2, x_3, x_4) = (0, 1, 0, 0, 1). \tag{9.2}$$

The decoder explicitly reverses the string:

```
bits = [int(bit) for bit in bitstring[::-1]]
```

This matters for every mapping from measurement to a real booking. It is not just a cosmetic choice in how a histogram is printed.

9.3 Validity and sampling metrics

The function `decode_bits()` checks the selected bookings independently of the penalty coefficients: exactly one booking per car and no shared station/slot. Let N be the final shot count, $n_{\boldsymbol{x}}$ the count for a string, and $\mathcal{F}$ the set of feasible strings. The empirical valid fraction is

$$\widehat{p}_{\text{valid}} = \frac{1}{N} \sum_{\boldsymbol{x} \in \mathcal{F}} n_{\boldsymbol{x}}. \tag{9.3}$$

This counts shots, not just distinct valid strings. A valid string observed 1,000 times contributes 1,000 to the numerator.

Displayed string	Shots	QUBO score	Interpretation
`00001`	719	79.70	A only; B unassigned
`10010`	663	46.30	A North late; B North early
`10100`	629	52.65	A South early; B North early
`11000`	629	56.65	A South late; B North early
`10001`	250	99.35	Both cars at North early

These are selected entries from the default run, not the complete histogram. The most frequent outcome was invalid. The three valid strings contributed $663 + 629 + 629 = 1921$ shots, giving

$$\widehat{p}_{\text{valid}} = \frac{1921}{4096} \approx 0.468994 = 46.90\%. \tag{9.4}$$

For reference, the uniform distribution has only three feasible strings among 32 and hence valid probability $3/32 = 9.375\%$. This comparison evaluates the distribution; it is not evidence of computational speedup over classical solvers.

10 Checkpoint 6: choose a sampled schedule and report it

The program selects the lowest booking score among feasible strings that were actually measured. It does not manufacture an unobserved string, repair an invalid sample, or invoke the reference exact solver as a fallback.

For the default run the selected logical vector was $(0, 1, 0, 0, 1)$:

$$\begin{aligned} \text{A}: &\ \text{North, reserved 09:00–10:00,} \\ \text{B}: &\ \text{North, reserved 08:00–09:00,} \\ \text{electricity bill} &= 10.40 + 18.40 = \text{Rs } 28.80, \\ \text{waiting} &= 65 + 5 = 70 \text{ minutes,} \\ \text{schedule score} &= 28.80 + 0.25(70) = 46.30. \end{aligned}$$

Actual charging finishes approximately at 09:56:44 for A and 08:50:11 for B. Both ports remain reserved until the end of their assigned slots.

Successful results use `status="feasible"`, not `"optimal"`. If no feasible string is measured, the result is `no_feasible_sample`; this does not prove that no feasible string exists. If preprocessing

finds a vehicle with no individual option, the CLI can report that definite obstruction before building a circuit. These are different kinds of failure.

The 33 tests in the current snapshot cover physical feasibility, QUBO penalties, all 32 basis-state energies, actual cost-gate phases, bit ordering, sampled-only selection, small Aer runs, and global conflict cases. The tests may enumerate tiny state spaces to verify correctness; the production quantum solver does not use those enumerations to select a schedule.

11 Experiments: what to improve and how to measure it

Change one setting at a time and compare several seeds. Record the best feasible sample score, valid fraction, best expected score, evaluation count, and runtime. If a run fails to sample any valid schedule, keep that failure in the results.

Setting	Changes	Interpretation
`-layers`	Number p of cost/mixer pairs	More gates and $2p$ angles; more expressiveness does not guarantee better training.
`-iterations`	Evaluation budget per restart	More opportunities to optimize; convergence may occur earlier.
`-restarts`	Number of angle initializations	Helps explore different local solutions; increases total work.
`-shots`	Final measurement repetitions	Changes sampling coverage, not ideal training precision or angles.
`-seed`	Initialization and final sampling randomness	Tests sensitivity to favorable random choices.
`-penalty`	Constraint coefficient P	Alters feasibility emphasis relative to operating costs.

```
# Inspect the simplest cost/mixer circuit.
.venv/bin/python main.py --layers 1 --show-circuit

# Increase expressiveness and training effort together.
.venv/bin/python main.py --layers 3 --iterations 300 --restarts 5

# Increase final sampling coverage only.
.venv/bin/python main.py --shots 16384

# Repeat with a different initialization and sampling seed.
.venv/bin/python main.py --seed 21
```

The returned `history`, `angles`, and `parameter_order` make learning-curve plots and angle comparisons possible. A later extension could change the mixer, initialize from a useful state, simulate device noise, or use finite-precision expectation estimates. These are experiments to implement; they are not features already enabled by the current CLI.

Larger shot counts improve the probability of observing an existing rare useful outcome. If its probability is $p_\star$ and shots are independent, the chance of seeing it at least once in N shots is

$$1 - (1 - p_\star)^N. \tag{11.1}$$

Shots cannot reveal an outcome with exactly zero probability. The validity fraction and training expectation tell us more than the best observed score alone.

12 Boundaries, hardware access, and the browser explainer

12.1 What the current physical model assumes

Each car needs one uninterrupted session within one nonoverlapping hour slot. Each station has one port. Consumption and charging power are constant, charging efficiency is 100%, and the driver departs as soon as available. Actual charging completion determines deadline compliance, but the reservation blocks the full slot. Waiting is the time at the station before the slot starts; driving has no separate objective penalty. Reserve is checked on arrival, not on a later onward journey. These choices simplify the application and must be revisited when the product requirements change.

12.2 Simulation versus a physical quantum processor

Qiskit builds the circuits; Aer simulates them locally. The IBM API key is not used by this program. A physical IBM processor would require a separate Runtime integration, backend selection, compilation to supported gates/connectivity, and a training/measurement workflow compatible with that backend. An API key alone does not change `AerSimulator()` into hardware execution.

An ideal statevector stores 2^n complex amplitudes. A compact Pauli operator representation does not remove that simulator memory cost. The current single-session allocation model is also approachable by strong classical assignment methods. This project demonstrates a quantum optimization workflow; it makes no claim that QAOA is necessary for this model or faster than classical computing.

12.3 The interactive parking illustration

The separate browser illustration in `web/parking-anatomy/` shows the same parking/charging example, waiting preferences, and a five-bit booking inspector. Build it with

```
node web/parking-anatomy/build.mjs
```

Then open `web/parking-anatomy/dist/index.html`. Its build exports sample bookings from Python and can use a saved fixture. The browser computes 32 illustrative QUBO energies classically. It does not run QAOA or connect to IBM; its exported baseline uses the reference classical solver. Run `main.py` to execute the actual Qiskit circuit. The browser's own README contains its preview instructions.

12.4 Code-to-concept map

Function	Responsibility
`sample_scenario`	Input dictionaries for cars, stations, slots, and travel.
`generate_options`	Physical feasibility, journey quantities, and booking scores.
`build_qubo`	Data-driven variables, exactly-one groups, and conflict penalties.
`qubo_value`	Evaluate a supplied bit vector; no optimization.
`qubo_to_ising`	Collect Z/ZZ coefficients, offset, and normalization.
`build_qaoa_circuit`	Explicit H, RZ, RZZ, and RX operations.
`solve_qaoa`	Train angles on Aer and measure the final circuit.
`decode_bits`	Independently check selected booking constraints.
`summarize_counts`	Reverse display order and select a feasible measured result.

The main quantum implementation is `src/quantum_scheduler.py`; ordinary physical calculations are in `src/ev_scheduler.py`. The CLI is in `main.py`.

A Notation and glossary

Use this appendix whenever a symbol interrupts your reading. An index identifies a particular object; it does not mean multiplication. Hats distinguish operators from ordinary numbers. Bold symbols collect several numbers into a vector.

Symbol or term	Meaning in this project
x_i	A binary decision: select booking i when one; leave it unselected when zero.
$\boldsymbol{x}$	Logical vector $(x_0, \ldots, x_{n-1})$ describing a complete proposed schedule.
q_i	Qubit carrying booking bit i when measured.
n	Number of booking variables/qubits in the direct encoding.
$\mathcal{V}$, G_v	Vehicle set and booking-index set for vehicle v.
$\mathcal{C}$	Unordered pairs of different-car bookings sharing a station and slot.
c_i, λ	Booking score and waiting weight, with default $\lambda = 0.25$ points/min.
P, U	Constraint penalty and bound on any feasible schedule's operating score.
C_0, L_i, Q_{ij}	Constant, linear, and upper-triangular pair coefficients in the expanded QUBO.
z_i	Numerical sign $(-1)^{x_i}$: $+1$ for bit zero, -1 for bit one.
$\widehat{Z}_i$	Pauli Z operator acting on qubit i; a matrix, not a binary variable.
$\widehat{X}_i$	Pauli X operator acting on qubit i; interchanges basis states zero and one.
$\mathbb{I}$	Identity operator; leaves a vector unchanged.
h_i, J_{ij}	Coefficients multiplying Z and ZZ in the Ising cost operator.
C_I	Constant identity coefficient after converting to Ising form; called `offset`.
$\widehat{H}_f$	Full cost operator $C_I\mathbb{I} + \widehat{H}_C$; its eigenvalue on a booking basis state is $f(\boldsymbol{x})$.
$\widehat{H}_C$	Nonconstant cost Hamiltonian used to construct gates.
$s, \widetilde{H}_C$	Positive normalization scale and normalized nonconstant cost $\widehat{H}_C/s$.
H_{Had}	Hadamard gate. Different from the cost Hamiltonian despite the shared letter H.
$\lvert\psi\rangle$, $\langle\psi\rvert$	State vector (ket) and conjugate-transposed row vector (bra).
$a_{\boldsymbol{x}}$	Complex amplitude of outcome $\boldsymbol{x}$; its squared magnitude is a probability.
$\langle u \mid v\rangle$	Inner product of vectors, yielding a complex scalar.
$\langle\psi\rvert\widehat{A}\lvert\psi\rangle$	Expectation of operator $\widehat{A}$ in normalized state $\lvert\psi\rangle$.
i	Imaginary unit, $\mathrm{i}^2 = -1$; distinct from an italic index i.
ϕ, θ	Phase or rotation angles, measured in radians.
γ_ℓ, β_ℓ	Cost and mixer parameters for layer ℓ, pronounced gamma and beta.
p, ℓ	Number of QAOA layers and the current layer index.
U_C, U_M	Cost and mixer gate operations, each a unitary operator.
R_X, R_Z, R_{ZZ}	X rotation, Z rotation, and two-qubit ZZ rotation.

Symbol or term	Meaning in this project
$\otimes$	Tensor product: combines state spaces or operators for multiple qubits.
$\sum$, $\prod$	Add or multiply the indexed quantities. In operator products the rightmost acts first.
Pr, $\mathbb{E}$	Probability and expectation (probability-weighted mean).
Basis state	A definite bit string, such as all qubits zero.
Superposition	A state expressed as a linear combination of basis states.
Phase	Direction of a complex amplitude; relative phases can affect later interference.
Global phase	The same unit-magnitude factor multiplying every amplitude; does not affect measurement probabilities.
Interference	Reinforcement or cancellation when operations combine amplitude contributions.
Eigenvalue	Number by which an operator multiplies a particular eigenvector.
Hamiltonian	A Hermitian operator. Here it represents optimization costs, not battery energy.
Ground state	An eigenstate with the smallest eigenvalue of a Hamiltonian.
Ansatz	The selected family of parameterized circuits; here repeated cost/mixer layers.
Shot	One circuit preparation, execution, and measurement.
Estimator	Computes an observable's expectation for a circuit. Our Aer estimator uses ideal precision zero.
Statevector	The array of 2^n complex amplitudes used by the ideal simulator.
COBYLA	Classical derivative-free optimizer that changes circuit angles.

B Mathematical background and gate calculations

B.1 Complex numbers and magnitudes

A complex number is $a = u + \mathrm{i}v$ with real u, v. Its conjugate is $a^* = u - \mathrm{i}v$, and

$$|a|^2 = a^* a = u^2 + v^2. \tag{B.1}$$

Euler's identity writes a unit-length phase factor as

$$e^{\mathrm{i}\phi} = \cos\phi + \mathrm{i}\sin\phi, \qquad |e^{\mathrm{i}\phi}| = 1. \tag{B.2}$$

Thus multiplying an amplitude by $e^{\mathrm{i}\phi}$ changes its direction but not its measurement probability. Multiplying *every* amplitude by the same phase leaves every future measurement probability unchanged under the same subsequent operations, since the common scalar factors through those linear operations.

For a normalized state $|\psi\rangle$, $\langle\psi \mid \psi\rangle = 1$. Its bra is the transpose with all entries conjugated. A Hermitian matrix satisfies $A^\dagger = A$ and has real eigenvalues. A unitary matrix satisfies $U^\dagger U = \mathbb{I}$ and preserves normalization. Hamiltonians and gates serve different roles even though both are operators.

B.2 Relative phase can be revealed by a later gate

Define

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}. \tag{B.3}$$

Both have immediate measurement probabilities $1/2, 1/2$. But

$$H_{\mathrm{Had}}|+\rangle = |0\rangle, \qquad H_{\mathrm{Had}}|-\rangle = |1\rangle. \tag{B.4}$$

For the first case, the new zero amplitude is $(1+1)/2 = 1$ and the new one amplitude is $(1-1)/2 = 0$. For the second, those reinforce/cancel roles reverse. This demonstrates interference without claiming that phase alone changes the probabilities of an immediate computational-basis measurement.

B.3 Tensor products and Qiskit's basis order

Two independent states combine as $|a\rangle \otimes |b\rangle$, usually abbreviated $|ab\rangle$. Their joint vector has four entries. For two qubits, conventional Qiskit displayed basis order is

$$|q_1 q_0\rangle \in \{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}. \tag{B.5}$$

Hence $\widehat{Z}_0 = \mathbb{I} \otimes \widehat{Z}$ and $\widehat{Z}_1 = \widehat{Z} \otimes \mathbb{I}$ in that order. For a logical vector $\boldsymbol{x} = (x_0, \ldots, x_{n-1})$, its statevector array index is

$$k = \sum_{i=0}^{n-1} 2^i x_i. \tag{B.6}$$

For $(0, 1, 0, 0, 1)$, $k = 2 + 16 = 18$, displayed in binary as `10010`.

B.4 Matrix forms of the rotations

For an angle θ, the gates used here are

$$R_Z(\theta) = \begin{pmatrix} e^{-\mathrm{i}\theta/2} & 0 \\ 0 & e^{+\mathrm{i}\theta/2} \end{pmatrix}, \tag{B.7}$$

$$R_X(\theta) = \begin{pmatrix} \cos(\theta/2) & -\mathrm{i}\sin(\theta/2) \\ -\mathrm{i}\sin(\theta/2) & \cos(\theta/2) \end{pmatrix}, \tag{B.8}$$

$$R_{ZZ}(\theta) = \mathrm{diag}\left(e^{-\mathrm{i}\theta/2}, e^{+\mathrm{i}\theta/2}, e^{+\mathrm{i}\theta/2}, e^{-\mathrm{i}\theta/2}\right). \tag{B.9}$$

The RZZ diagonal is ordered as $00, 01, 10, 11$. Equal bits have ZZ eigenvalue $+1$, while different bits have eigenvalue -1. The original $x_i x_j$ penalty is not represented by RZZ alone: its converted Z terms and identity offset also contribute. Together they reproduce a penalty only on 11.

B.5 The one-qubit phase/mixer example, fully worked

After H and $R_Z(-\pi/2)$ the state is

$$|\psi_1\rangle = \frac{1}{\sqrt{2}}\left(e^{\mathrm{i}\pi/4}|0\rangle + e^{-\mathrm{i}\pi/4}|1\rangle\right). \tag{B.10}$$

Both amplitudes still have magnitude $1/\sqrt{2}$. The next gate is

$$R_X(-\pi/2) = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & \mathrm{i} \\ \mathrm{i} & 1 \end{pmatrix}. \tag{B.11}$$

The resulting amplitudes are

$$a_0 = \frac{e^{\mathrm{i}\pi/4} + \mathrm{i}e^{-\mathrm{i}\pi/4}}{2} = e^{\mathrm{i}\pi/4}, \tag{B.12}$$

$$a_1 = \frac{\mathrm{i}e^{\mathrm{i}\pi/4} + e^{-\mathrm{i}\pi/4}}{2} = 0. \tag{B.13}$$

The final state is $e^{\mathrm{i}\pi/4}|0\rangle$, with the same measurement probabilities as $|0\rangle$. The first amplitude reinforces while the second cancels. The remaining global phase is unobservable in the final probabilities.

```
from math import pi
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

qc = QuantumCircuit(1)
qc.h(0)
print(Statevector.from_instruction(qc).probabilities_dict())
qc.rz(-pi / 2, 0)
print(Statevector.from_instruction(qc).probabilities_dict())
qc.rx(-pi / 2, 0)
print(Statevector.from_instruction(qc).probabilities_dict())
```

Floating-point output may show a tiny nonzero value such as 10^{-32} instead of exact zero. This is numerical roundoff, not a meaningful probability of failure.

C Full numerical audit of the sample objective

C.1 Expanded QUBO coefficients

At $P = 57.65$, the generated coefficients are

$$C_0 = 115.30, \tag{C.1}$$
$$(L_0, L_1, L_2, L_3, L_4) = (-35.60, -31.00, -24.65, -20.65, -38.00), \tag{C.2}$$
$$Q_{ij} = 115.30 \quad (0 \le i < j \le 3), \tag{C.3}$$
$$Q_{04} = 57.65. \tag{C.4}$$

All omitted pairs have coefficient zero. In full,

$$\begin{aligned} f(\boldsymbol{x}) = 115.30 &- 35.60x_0 - 31x_1 - 24.65x_2 - 20.65x_3 - 38x_4 \\ &+ 115.30(x_0x_1 + x_0x_2 + x_0x_3 + x_1x_2 + x_1x_3 + x_2x_3) \\ &+ 57.65x_0x_4. \end{aligned} \tag{C.5}$$

This expression and Equation (4.2) are identical on binary inputs. The negative coefficients arise from algebraically expanded penalties.

C.2 Derive one Z coefficient and one pair coefficient

For qubit zero, incident pair coefficients are $Q_{01}, Q_{02}, Q_{03}, Q_{04}$. Therefore

$$\begin{aligned} h_0 &= -\frac{-35.60}{2} - \frac{115.30 + 115.30 + 115.30 + 57.65}{4} \\ &= 17.80 - 100.8875 = -83.0875. \end{aligned} \tag{C.6}$$

The A–B conflict yields $J_{04} = Q_{04}/4 = 14.4125$. The identity offset is

$$\begin{aligned} C_I &= C_0 + \frac{1}{2}\sum_i L_i + \frac{1}{4}\sum_{i<j} Q_{ij} \\ &= 115.30 - 74.95 + 187.3625 = 227.7125. \end{aligned} \tag{C.7}$$

C.3 Verify the selected schedule three ways

For $\boldsymbol{x} = (0, 1, 0, 0, 1)$:

1. From the factored scheduling expression: all penalties vanish and $f = 26.65 + 19.65 = 46.30$.
2. From the expanded QUBO: only x_1, x_4 are one, with no pair between them, so $f = 115.30 - 31.00 - 38.00 = 46.30$.
3. From Ising signs: $\boldsymbol{z} = (+1, -1, +1, +1, -1)$. The Z sum is -167; the six A–A ZZ products sum to zero; the A–B ZZ term is -14.4125. Thus $E_C = -181.4125$ and $f = C_I + E_C = 46.30$.

All representations assign the same score. This is the invariant tested by checking every one of the 32 basis states.

C.4 General coefficient generation and a matrix convention trap

For each vehicle group the compiler adds P to the constant, $-P$ to every member's linear coefficient, and $2P$ to every within-group pair. Different-car resource conflicts add P to their pairs. Thus new data changes the coefficients without a new hand-written equation.

The code stores each pair once, with $i < j$. If instead writing a symmetric matrix expression $\boldsymbol{x}^\mathsf{T} M \boldsymbol{x}$, remember that M_{ij} and M_{ji} both contribute. For this convention choose $M_{ii} = L_i$ and $M_{ij} = M_{ji} = Q_{ij}/2$. Copying upper-triangular coefficients into both halves without halving would double every pair cost.

D Exercises and common points of confusion

Try these before reading Appendix E.

1. A new car C has options x_5, x_6. Write its exactly-one penalty. Suppose x_1 and x_5 are different cars using the same station and slot; write the additional conflict term.
2. For A, what are the exactly-one penalties when zero, one, two, or three options are selected? Why does "all bits zero" fail?
3. Evaluate the sample objective at $(1, 0, 0, 0, 1)$ using $P = 57.65$. Which penalties vanish and which does not?
4. A drives an additional 3 km at 0.2 kWh/km with the same battery target. How much extra charging energy is needed?
5. If $x = 1$, what is z? Convert $f(x) = 8x$ into a Z-form cost operator and verify both bit-state scores.
6. Expand $P(a + b - 1)^2$ using binary identities. Why is the coefficient of ab equal to $2P$, rather than P?
7. After a uniform initial state and only diagonal cost gates, what are the five-qubit measurement probabilities? Why is the mixer needed?
8. A Z term has coefficient $h = -3$, scale $s = 6$, and $\gamma = 0.4$. What RZ angle should the code use? What RX angle corresponds to $\beta = 0.2$?
9. Decode Qiskit's displayed string `10100` into logical booking order and identify the selected bookings.
10. Does increasing `-shots` change the current exact training objective or the learned angles when all training settings and the seed stay fixed?
11. Why can a run's best valid sample score be 46.30 while its expected QUBO score is about 73.88? What does the invalid mode `00001` omit?
12. If no valid schedule is sampled, can we conclude that the mathematical problem is infeasible? What is different if a car has no individual option?

D.1 Interpretations to keep separate

A variable is a booking, not a vehicle. Four of the sample bits belong to A, but the exactly-one constraint allows only one of those four to be selected.

Joint planning is different from simultaneous charging. The optimizer selects a complete schedule for all cars. Resource constraints determine which cars may physically charge at the same time.

A penalty is a scoring device. The implemented mixer can visit invalid strings. A penalty discourages them but does not make their probability zero. The final decoder is still necessary.

A cost Hamiltonian is not a measurement of electricity. Its eigenvalues are mathematical scores. Its constant, signs, and units need not resemble battery energy or monetary spending.

Superposition does not expose all answers at once. Each measurement returns one string. QAOA tries to shape their distribution through interference.

Correct encoding does not imply successful optimization. Tests can prove that an objective matches rules on a small instance while the optimizer still trains poorly. Evaluate encoding correctness and search quality separately.

E Exercise answers

1. Add $P(x_5 + x_6 - 1)^2$ for C and Px_1x_5 for that resource conflict. The builder generates both from the relevant group and pair data.
2. The penalties are $P, 0, P, 4P$. All-zero bits leave cars unassigned; cheap electricity spending alone would otherwise favor serving nobody.
3. Each car has exactly one booking, so its group penalty is zero. Both use North early, so the conflict penalty is P. The score is $22.05 + 19.65 + 57.65 = 99.35$.
4. Extra consumption is $3(0.2) = 0.6\,\text{kWh}$. Arrival energy falls by that amount, so charging energy increases by $0.6\,\text{kWh}$.
5. $z = -1$. The operator is $4\mathbb{I} - 4\widehat{Z}$, giving $4 - 4 = 0$ on bit zero and $4 + 4 = 8$ on bit one.
6. $P(a + b - 1)^2 = P - Pa - Pb + 2Pab$. The cross term in the square is $2ab$; binary identities reduce a^2, b^2 but do not remove that factor of two.
7. Each probability remains $1/32$. Cost gates multiply each amplitude by a unit-magnitude phase. Mixing combines contributions with different phases and can change the magnitudes, hence probabilities.
8. The RZ angle is $2\gamma h/s = 2(0.4)(-3)/6 = -0.4$ radians. The RX angle is $2\beta = 0.4$ radians. The two rotations have different axes and roles.
9. Reverse the displayed string to get $(0, 0, 1, 0, 1)$. Select A at South early and B at North early, with booking score $33 + 19.65 = 52.65$.
10. No. In this implementation the training expectation is ideal and independent of final shots. Increasing shots changes the final histogram and may reveal previously unobserved outcomes.
11. The expectation averages every outcome, including costly invalid ones. The best sample describes one selected valid schedule. `00001` selects only x_0, giving A a booking but none to B.
12. No sampled valid outcome does not prove infeasibility. In contrast, an empty option group proves that no complete schedule can serve that vehicle under the current model and supplied inputs.

F Commands, defaults, and reproducibility

CLI setting	Default
`-layers`	2
`-iterations`	160 per restart
`-restarts`	3
`-shots`	4096
`-seed`	7
`-penalty`	automatic $U + 1$
`-show-circuit`	off

The seed fixes initial-angle generation and final simulator sampling for the current implementation. Optimizer and simulator changes across library versions can change results even with the same seed. The recorded histogram in this guide is an observed snapshot, not a permanent expected output specification.

Training requires positive integer layers, shots, evaluation budget, and restarts; the budget must be at least $2p+2$. The seed must be a nonnegative integer and a custom penalty must be positive and finite. Larger scenarios increase qubit requirements; a small number of vehicles does not by itself guarantee a small quantum circuit if each vehicle has many possible bookings.

For experiments, record a row containing

$$\begin{aligned} \text{settings} &= (p,\ \text{budget},\ \text{restarts},\ N,\ \text{seed},\ P), \\ \text{results} &= (\widehat{p}_{\text{valid}},\ \text{best sample score},\ F_{\text{best}},\ \text{runtime}). \end{aligned}$$

Keep electricity bill and waiting separately as well. If you change the waiting weight, the definition of the score changes; scores from different weights are not directly comparable as if they measured the same preference.

References

[1] IBM Quantum Documentation, *Quantum approximate optimization algorithm.* https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm. Reference for the QUBO-to-Hamiltonian and hybrid QAOA workflow.

[2] IBM Quantum Documentation, *RZGate.* https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.RZGate. Reference for the rotation convention $R_Z(\theta) = e^{-\mathrm{i}\theta Z/2}$.

[3] IBM Quantum Documentation, *RZZGate.* https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.RZZGate. Reference for the two-qubit ZZ rotation convention and diagonal form.

[4] Qiskit Aer Documentation, *EstimatorV2.* https://qiskit.github.io/qiskit-aer/stubs/qiskit_aer.primitives.EstimatorV2.html. Reference for the simulator primitive used during training.

[5] SciPy Documentation, *minimize(method='COBYLA').* https://docs.scipy.org/doc/scipy/reference/optimize.minimize-cobyla.html. Reference for the optimizer options and evaluation budget.